

Lab2 类的深入剖析 (part-II)

学习目标:

1. 掌握 `const` 对象和 `const` 成员函数的声明与使用限制。
2. 深刻理解包含对象成员时，构造函数（含成员初始化列表）和析构函数的调用顺序。
3. 掌握类的组合用法及常数据成员的初始化。

实验

实验一：（美琴与硬币）

1. 问题描述

创建一个硬币类 `Coin` 和一个超能力者类 `Esper`，模拟美琴发射“超电磁炮”的过程，并观察类的生命周期。

- **Coin 类**：包含一个 `private` 的整型数据 `value`（面值）。编写带参数的构造函数和析构函数。在构造函数中打印 `"Coin [value] is created."`，在析构函数中打印 `"Coin [value] is destroyed."`。
- **Esper 类**：包含一个 `private` 的字符串类型 `name`（名字，默认为"`Misaka Mikoto`") 和一个 `Coin` 类的对象成员 `projectile`。
 - o 提供一个构造函数，必须使用成员初始化列表来初始化 `projectile` 和 `name`，并在构造函数体中打印 `"Esper [name] is ready."`。
 - o 提供析构函数，打印 `"Esper [name] has left."`。
 - o 提供一个 `public` 成员函数 `fire()`，打印发射台词（见输出示例）。
- 在 **main 函数**中，利用代码块 `{ }` 创建一个 `Esper` 对象，观察并验证对象成员与宿主对象构造和析构的先后顺序。

2. 输出示例

```
=====
Coin 100 is created.
Esper Misaka Mikoto is ready.
Fire! Railgun!
Esper Misaka Mikoto has left.
Coin 100 is destroyed.
=====
```

实验二：（大黄蜂的丝与针）

1. 问题描述

在《空洞骑士：丝之歌》中，主角大黄蜂（Hornet）使用丝线（Silk）和针（Needle）进行战斗。请创建一个 **Hornet** 类。

- 包含两个 **private** 数据成员：**int** `currentSilk`（当前丝线量）和常数据成员 **const int** `maxSilk`（最大丝线量）。

- 提供一个构造函数，使用成员初始化器语法将 `maxSilk` 初始化为传入的参数（若不传默认设为 100），将 `currentSilk` 初始化为 0。

- 提供以下 **public** 成员函数：

- o `gatherSilk(int amount)`: 非 **const** 函数，增加丝线量，但不超过 `maxSilk`。

- o `heal()`: 非 **const** 函数，消耗 30 点丝线回血。如果丝线不足，打印 "Not enough silk!"。

- o `displayStatus() const`: **const** 成员函数，以 `Silk: [current]/[max]` 的格式打印当前状态。

- 在 **main** 函数中，创建一个普通的 **Hornet** 对象进行收集和回血操作；再创建一个 **const Hornet** 对象，验证它只能调用 `displayStatus() const` 函数，若尝试调用 `heal()` 会在编译期报错。

2. 输出示例

```
Normal Hornet action:
Silk: 50/100
Heal successful.
Silk: 20/100
```

```
Const Hornet action:
Silk: 0/150
```

实验三：（里昂与物资管理）

在“卫戍协议”模式中，你需要部署干员（Operator）到战术建筑（TacticalBuilding）中。

- Operator** 类：包含 **private** 数据成员 `name`（干员代号，字符串）和 `atk`（攻

击力，浮点数）。提供构造函数（需有默认值），并提供一个 `getAtk() const` 函数获取攻击力，以及一个 `getName() const` 函数。

• **TacticalBuilding 类:**

- o 包含以下 `private` 成员：一个 `const int buildingID`（建筑编号），一个 `Operator` 类的对象成员 `stationedOp`（驻守干员），以及一个增益倍率 `double buffMultiplier`。

- o 提供一个构造函数，接收干员名字、干员攻击力、建筑编号和增益倍率。必须通过成员初始化列表来初始化 `buildingID` 和 `stationedOp` 对象。

- o 提供一个 `const` 成员函数 `calculateTotalDamage() const`，返回该建筑的实际总伤害（计算公式：干员攻击力 * 增益倍率）。

- o 提供一个 `printInfo() const` 函数，按指定格式输出建筑和干员信息。

2. 输出示例

```
Building ID: 101
Stationed Operator: Liskarm
Base ATK: 450.0
Buff Multiplier: 1.5
Total Output Damage: 675.0
-----
Building ID: 102
Stationed Operator: Mudrock
Base ATK: 880.0
Buff Multiplier: 1.2
Total Output Damage: 1056.0
```

实验四：（里昂与物资管理）

1. 问题描述

在《生化危机 9》中，主角里昂（Leon）需要管理他随身携带的武器（Weapon）。本题要求综合运用类的组合、`const` 特性及生命周期管理。

• **Weapon 类:** 包含 `weaponName`（字符串）和 `const int maxAmmo`（最大载弹量）。

- o 构造函数初始化这两个值，并打印 "Weapon [weaponName] loaded."。

- o 析构函数打印 "Weapon [weaponName] dropped."。

- o 提供 const 函数 `getWeaponInfo()` const 返回武器名。
- **Leon 类:**
 - o 包含一个常成员变量 `const string codeName` (代号)。
 - o 包含两个 `Weapon` 对象成员: `primary` (主武器) 和 `secondary` (副武器)。
 - o 构造函数接收代号和两把武器的名称及弹匣量。要求思考并决定初始化列表中 `primary` 和 `secondary` 的书写顺序。构造函数体内打印 "Agent [codeName] deployed."。
 - o 析构函数打印 "Agent [codeName] extracted."。
- 在 **main 函数** 中, 实例化一个 `Leon` 对象, 要求认真观察控制台的输出顺序。

2. 输出示例

```
Weapon SilverGhost loaded.  
Weapon Shotgun loaded.  
Agent Kennedy deployed.  
--- Mission Ongoing ---  
Agent Kennedy extracted.  
Weapon Shotgun dropped.  
Weapon SilverGhost dropped.
```